Amphion

Software Design Document (SDD) · CSCI 5520 - AI-Driven Software Development

Student Name	@Elijah
Date	Oct 12, 2026
Document Version	v0.3
Project Repository / URL	github.com/etabor3096/Amphion

1 Problem & Users

1.1 Problem Description

Most people learn guitar by playing songs they already love, but beginners hit a wall: they do not yet have the ear training to work out a song by listening. Figuring out which notes or chords are being played, where they sit on the fretboard, and when the changes happen is a skill that takes years to develop. Until then, a beginner depends on someone else having written the song down as tablature (tabs), a simple notation that shows which string and fret to play.

That dependency breaks down often. Many songs have no tabs at all, including newer releases, songs by smaller or local artists, live arrangements, and church worship sets. When tabs do exist, they are usually written by volunteers and are frequently wrong: incorrect notes or chords, the wrong tuning or capo position, or timing that does not match the recording. An experienced player can spot and fix these mistakes by ear. A beginner cannot, so they practice the wrong part, cannot make it sound like the record, and often conclude the problem is their own ability.

The core problem is that beginner and early-intermediate guitar and bass players have no reliable way to turn a song they hear into an accurate, playable part when good tabs are missing or wrong, and they lack the ear skills to do it themselves.

1.2 Target Users & Stakeholders

User / Stakeholder	Relationship to the Problem
Beginner / Early-Intermediate Player (primary user)	Plays acoustic, electric, or bass guitar and wants to learn specific songs. Cannot yet transcribe by ear, so is blocked when tabs are missing and misled when tabs are wrong.
Guitar Teacher (secondary user)	Regularly gets song requests from students and must transcribe them by hand before a lesson. Needs a faster starting point that can be checked and corrected.

2 Proposed Solution & Scope

2.1 Proposed Solution

Amphion is an iOS app that listens to a song and turns it into guitar or bass tablature (tab) that a beginner can play along with. A user either records audio through the phone's microphone (a song playing nearby, or themselves playing) or uploads an audio file they own. Amphion separates the guitar and bass parts from the rest of the mix, analyzes them, and produces a tab: which string and fret to play for each note, lined up with the song's beats and bars. Chord names appear in a small line above the tab as a supporting guide.

The user then practices inside the app. When they press play, the song plays and the tab moves with it: a playhead sweeps across the tab, the notes being played light up, and the tab scrolls on its own so the current bar is always on screen. The user can slow the song down, loop a hard section, or tap any note to jump there, and the tab stays locked to the audio the whole time. Amphion also reports the song's key, tempo, and tuning so the user is set up correctly before they start. Where the analysis is unsure, Amphion says so instead of hiding it: low-confidence notes are flagged, and the user can correct them. Every transcription is saved to the user's personal library.

The expected value is that a beginner no longer depends on someone else having written an accurate tab. Any song they can play or record becomes something they can learn, and the honest confidence flags keep them from practicing mistakes without knowing it. A guitar teacher gets the same benefit as a fast first draft to check before a lesson.

2.2 Minimum Viable Product (MVP) Scope

The MVP proves one thing: Amphion can turn a recorded or uploaded song into beat-synced guitar or bass tablature that plays in sync with the song, so a beginner can practice with it. It includes:

- **Audio input:** record from the microphone or upload an audio file from the device.
- **Tab transcription:** separate the guitar and bass parts from the mix, detect the notes in each part, give every note a playable string and fret, and line notes up with the detected beat grid. Chord names are detected as a secondary layer shown above the tab.
- **Song details:** detect and display key, tempo (BPM), and tuning (how far the recording sits from standard pitch).
- **Synced tab playback:** the song plays with a moving playhead, highlighted current notes, and automatic scrolling, all driven by the audio's own playback position so the tab never drifts from the song; plus tap-to-jump, slow-down, and A-B looping (repeating a chosen section).
- **Confidence flags and editing:** low-confidence notes are visibly marked, and the user can change, add, or delete any note.
- **Accounts and personal library:** sign up, sign in, reset a forgotten password, manage the account (display name, email, password, account deletion), and save, reopen, rename, and delete transcriptions, organize them into folders, give each song its own artwork, and search, filter, and sort the library.
- **Supporting capability:** a processing service that runs the analysis without the user keeping the app open on one screen, and cloud storage for accounts and saved transcription results (song audio stays on the phone; see RC-03).

The MVP accepts songs up to 3 minutes long (RC-07); supporting full-length songs is a goal by the end of the project (OQ-04).

2.3 Out of Scope / Non-Goals

Not part of this project:

- Real-time transcription. Results appear after processing finishes, not live while the song plays.
- Sharing or a public tab library. Transcriptions stay private to the user who made them; there is no posting, publishing, or exchanging tabs between users.

- Importing songs from streaming links (YouTube, Spotify, Apple Music, and similar). Audio comes only from the microphone or files on the user's device.
- Payments, subscriptions, and paid tiers. A paid model is a possible future business direction, not part of this project.
- Android, web, and desktop clients. The project targets iOS only; Android is a possible later effort.

Deferred until after the MVP (stretch goals, not to be built unless the MVP is complete):

- Guitar techniques beyond plain notes in the tab (bends, slides, hammer-ons, pull-offs, vibrato), and standard music notation.
- Separate tabs for several guitar parts in one song (for example rhythm and lead), and playing along with the guitar or bass part muted. The MVP shows one combined guitar tab (RISK-06).
- Beginner-simplified fingerings and a simplified/full difficulty toggle.
- Capo suggestions. Audio cannot reveal whether a capo was used, so this would be a suggested capo position that gives easier chord shapes, not a detected one.
- Instruments beyond acoustic, electric, and bass guitar (for example piano, drums, or vocals).

3 User Stories & Acceptance Criteria

The stories below define the major MVP behavior. "Player" means the primary user; a guitar teacher uses the same stories. Stable IDs let each story be traced into the UI/UX design, implementation plan, GitHub Issues, and tests.

3.1 Account & Library

US-01 - Create Account and Sign In

User Story: As a player, I can create an account and sign in with Apple, Google, or email and password so that my transcriptions are saved to me.

Pass/Fail Check: Pass if each of the three sign-in methods creates an account and reaches the library screen, and signing out then back in shows the same library; fail if any method errors or a wrong password grants access. An email/password user who forgot their password can request a reset email from the sign-in screen and then sign in with the new password.

US-02 - Save Transcriptions to My Library

User Story: As a player, I can have every finished transcription saved to my personal library so that I can come back to it later.

Pass/Fail Check: Pass if a finished transcription appears in the library with its title and date and reopens with the same tab, chord names, and edits after the app is closed and relaunched.

US-03 - Manage My Library

User Story: As a player, I can rename and delete saved transcriptions so that my library stays organized.

Pass/Fail Check: Pass if a rename persists after relaunch and a delete (after confirmation) removes the song and its audio so it no longer appears or opens.

US-15 - Manage My Account

User Story: As a player, I can view my sign-in method, edit my display name, change my email or password, and delete my account so that I stay in control of my account and data.

Pass/Fail Check: Pass if a display-name change persists after relaunch; an email/password user can change their password and email only after re-entering their current password, and a new email takes effect only after the verification link is opened; Apple and Google users see their email as managed by that provider with no password option; and deleting the account (after typed confirmation) signs the user out, removes all their transcriptions, and the old credentials can no longer sign in.

US-16 - Organize Songs into Folders

User Story: As a player or teacher, I can create folders (for example, "Worship Set" or "Student: Maya") and move songs into them so that I can keep related songs together as my library grows.

Pass/Fail Check: Pass if the user can create, rename, and delete a folder; move a song into a folder or back out to All Songs; and see each folder's song count. A song belongs to at most one folder. Deleting a folder (after confirmation) moves its songs back to All Songs and never deletes them. All folder changes persist after relaunch.

US-17 - Search, Filter, and Sort My Library

User Story: As a player, I can search my library and open a filter menu to narrow it by date saved, song name, and artist, and sort the results, so that I can quickly find the song I want to practice.

Pass/Fail Check: Pass if typing in the search field shows only songs whose title or artist contains the text (ignoring capitalization); the filter menu can limit results by date saved (Last 7 days, Last 30 days, or a custom date range) and by one or more artists; sorting works by Newest, Oldest, Song name A-Z, and Artist A-Z; active filters appear as removable chips with a Clear all option; and a search with no matches shows a clear "No songs match" message instead of an empty screen.

US-18 - Set Song Artwork

User Story: As a player, I can choose a photo from my library or take a new one to use as a song's artwork, and see it large at the top of the Song screen, so that my library is easy to scan and each song feels like my own.

Pass/Fail Check: Pass if every song shows generated default artwork until the user changes it; a "Change Photo" button on the Song screen lets the user pick from Photos, take a photo, or reset to the default, with a square crop step; the new artwork appears on both the Song screen header and the Library card immediately and after relaunch, including on another signed-in iPhone; and an unsupported or oversized image is rejected with a clear message.

3.2 Audio Input & Processing

US-04 - Record a Song with the Microphone

User Story: As a player, I can record a song playing nearby, or myself playing, with my phone's microphone so that I can transcribe music that is not saved as a file.

Pass/Fail Check: Pass if the user can start, stop, and replay a recording of up to 3 minutes and submit it for transcription; recording stops automatically at 3 minutes.

US-05 - Upload an Audio File

User Story: As a player, I can upload an audio file from my device so that I can transcribe songs I already have.

Pass/Fail Check: Pass if MP3, M4A, and WAV files up to 3 minutes are accepted and submitted; fail if an unsupported or over-length file is accepted without a clear message explaining why it was rejected.

US-06 - Track Processing Status

User Story: As a player, I can see that my song is being processed and get notified when it is done so that I do not have to watch the screen.

Pass/Fail Check: Pass if a submitted song shows a Processing state in the library, changes to Ready when finished, and the user receives a notification when the app is in the background; a failed job shows a Failed state with an option to retry.

3.3 Transcription Results

US-07 - View Beat-Synced Tablature

User Story: As a player, I can see the song as tablature laid out on its beats and bars so that I know which string and fret to play and exactly when.

Pass/Fail Check: Pass if the tab shows six string lines for guitar (four for bass) labeled with the string names; bar lines come from the detected beat grid; every note sits at its detected time with its fret number on the correct string; notes played together are stacked in one column; chord names appear in a separate line above the tab wherever a chord was detected; and tapping any note or bar jumps playback to that moment.

US-08 - Play the Tab on My Instrument

User Story: As a player of acoustic guitar, electric guitar, or bass, I can switch between a guitar tab (covering both acoustic and electric guitar, which use the same six-string tab) and a bass tab, each made from that instrument's own part of the song with fret positions a beginner can reach, so that I can learn the part for the instrument I play.

Pass/Fail Check: Pass if the instrument setting switches the Song screen between the guitar tab and the bass tab; no two notes that sound together are placed on the same string; notes that sound together sit within a four-fret span wherever the instrument allows; open strings and lower frets are preferred when a note can be played in more than one place; fret numbers stay correct for the detected tuning offset; and tapping a chord name above the tab shows that chord's finger diagram. A song with no detectable bass part shows a clear "No bass part found" message instead of an empty tab.

US-09 - See Key, Tempo, and Tuning

User Story: As a player, I can see the song's key, tempo, and tuning so that I can set up my instrument correctly before playing.

Pass/Fail Check: Pass if all three values are shown on the song screen for every finished transcription, with any value the system could not determine marked as "Unknown" rather than guessed silently.

US-10 - See Where Amphion Is Unsure

User Story: As a player, I can see which notes Amphion is not confident about so that I know which parts to double-check by ear.

Pass/Fail Check: Pass if notes below the confidence threshold are visibly marked with an indicator that does not rely on color alone (for example, a dotted outline and a "?"), chord names below their threshold are marked the same way, and a song with no low-confidence notes shows no markers.

US-11 - Correct a Note

User Story: As a player or teacher, I can change, add, or delete notes in the tab so that the transcription matches what I hear.

Pass/Fail Check: Pass if the user can select a note and change its fret, move it to another string at the same pitch, or delete it; can add a note on any string at a beat position; can still correct a chord name from a chord picker; edits can be undone during the session and persist after relaunch; and an edited note no longer shows a low-confidence marker.

3.4 Practice Playback

US-12 - Play Along with Synced Tabs

User Story: As a player, I can play the song and watch a playhead move through the tab in time with the audio, with the notes being played highlighted and the tab scrolling on its own, so that I can play along without losing my place.

Pass/Fail Check: Pass if the playhead and highlighted notes stay within 50 ms of what is heard through the iPhone speaker or wired headphones, and stay visibly in sync on Bluetooth headphones once their output delay is corrected for (RC-18); sync holds after pausing, scrubbing, tapping a note to jump, changing speed, and every time a loop restarts; the tab scrolls on its own to keep the playhead on screen; if the user scrolls away by hand, auto-follow pauses and a "Back to playhead" button returns to it; the screen does not auto-lock while a song is playing; and a song whose audio is not on this iPhone shows the tab with a message that synced playback is available only on the device that recorded or uploaded it (OQ-03).

US-13 - Slow the Song Down

User Story: As a player, I can slow the song down without changing its pitch so that I can practice hard parts at a speed I can manage.

Pass/Fail Check: Pass if playback speed can be set from 50% to 100%, pitch stays the same at every speed, and the tab playhead stays in sync (US-12).

US-14 - Loop a Section

User Story: As a player, I can set a start point (A) and an end point (B) and loop that section so that I can repeat a tricky part until I learn it.

Pass/Fail Check: Pass if playback repeats from A to B continuously until the loop is cleared, and the loop works at any playback speed.

4 Requirements, Constraints & Design Rules

These rules apply across all user stories and must be respected by every future implementer, human or AI.

Security & Privacy

RC-01 - Authentication: All functionality except the sign-in screens requires a signed-in account. Supported methods are Sign in with Apple, Google sign-in, and email/password. The backend must verify the user's identity on every request rather than trusting the app.

RC-02 - Per-User Data Isolation: A user can read, edit, or delete only their own transcriptions and account data. No request may return or change another user's data.

RC-03 - Audio Retention: Song audio is uploaded only for processing and must be deleted from cloud storage as soon as the processing job finishes or fails. The separated guitar and bass parts made from it exist only inside the running job and are never stored. The cloud keeps only the transcription results (notes with their string and fret, beats, chord names, song details, edits) and any custom song artwork (DD-13). The phone keeps its local copy of the audio for playback.

RC-04 - Private by Design: Transcriptions are private to their owner. The MVP must not include any feature that publishes, shares, or exports transcriptions or audio to other users.

RC-05 - Microphone Use: The app records only after the user taps Record and stops when the user taps Stop or the length limit is reached. There is no background or always-on listening. The microphone permission prompt must explain why access is needed.

RC-06 - Secrets & Credentials: API keys, service credentials, and other secrets must never be embedded in the iOS app or committed to the repository.

RC-15 - Account Deletion: Users must be able to delete their account from inside the app (Settings > Account), as Apple requires for any app that supports account creation. Deletion requires confirmation and recent sign-in, removes the user's authentication

record, every Firestore record they own, and their stored song artwork, and cannot be undone.

RC-16 - Terms & Privacy: Settings links to a Terms of Use and a Privacy Policy. The Terms state that users must have the right to use any audio they record or upload (RISK-05).

RC-17 - Song Artwork Images: Artwork accepts JPEG, PNG, and HEIC photos. The app crops it to a square and compresses it to at most 1024 x 1024 pixels and 1 MB before upload. Choosing an existing photo uses the system photo picker, so the app never asks for access to the whole photo library; taking a new photo asks for camera permission with a plain explanation, only when the user chooses that option. Artwork is stored privately under the owner's account (RC-02) and is deleted when its song or the account is deleted (RC-15).

Performance & Reliability

RC-07 - Input Limits: The MVP accepts MP3, M4A, and WAV audio up to 3 minutes long. Limits are checked in the app before upload and enforced again by the backend.

RC-08 - Processing Time: Under normal network conditions, a 3-minute song must finish processing in under 2 minutes from upload to Ready.

RC-09 - Failure Handling: A failed or timed-out job must show a Failed state with a retry option. The app must never show a partial or empty tab as if it were complete.

RC-10 - Accuracy Benchmark: The project keeps a reference set of at least 10 songs with hand-checked guitar tabs (and bass tabs where the song has a bass part), covering both uploaded files and phone-mic recordings. Note accuracy is measured against this set whenever the transcription pipeline changes, so improvements and regressions are visible. A detected note counts as correct when its pitch matches a reference note and it starts within 50 ms of it; string and fret choices are scored separately. Chord-name accuracy is still measured for the fallback (RISK-01). The pass thresholds are tracked in Section 7.2.

RC-18 - One Clock for Sync: Every note, beat, bar, and chord time is stored in seconds from the start of the song's audio file (song time). During playback, the Song screen reads the current song time from the audio player itself on every screen refresh and draws the playhead and highlights from it; it never runs its own timer. Speed changes and loops change only how fast song time moves, so the stored tab never has to change. The display corrects for the device's audio output delay (for example, Bluetooth headphones) so the highlighted note matches what the user hears.

RC-19 - Same Audio for Analysis and Playback: The app uploads exactly the audio file it keeps on the phone for playback, with no trimming or re-encoding after upload, so the times the pipeline produces line up with the local file. The song record stores the file's duration and a content fingerprint (hash). If the local file is missing or does not match, the Song screen shows the tab without synced playback (OQ-03) rather than playing out of sync.

RC-20 - Tab Data Format: For each instrument, the pipeline stores a list of notes with start time, length, string, fret, pitch (MIDI note number), confidence, and whether the user edited it, plus the beat and downbeat times and the chord-name segments. Results are written to Firestore in chunks by bar range so no single record approaches Firestore's 1 MB document limit as songs grow toward full length (OQ-04).

Accessibility & User Experience

RC-11 - Honest Output: Amphion must never present uncertain results as certain. Notes and chord names below the confidence threshold are always marked, and any song detail that could not be determined is shown as "Unknown" rather than guessed.

RC-12 - Accessibility: All controls have VoiceOver labels, text respects Dynamic Type sizes, confidence markers and the current-note highlight do not rely on color alone, the tab can be read by VoiceOver bar by bar (for example, "Bar 3, beat 1: A string, fret 2"), and each chord diagram has a text alternative (for example, "G major, open position").

RC-13 - Beginner-Friendly Language: User-facing text uses plain musical terms a beginner can follow and briefly explains terms such as tab, BPM, and tuning offset where they appear. Machine-learning or internal technical terms are never shown to users.

Platform & Compatibility

RC-14 - Supported Devices: The MVP targets iPhone on iOS 17 or later. iPad layouts, Android, web, and desktop are not required.

5 Platform & Architecture

5.1 System Components & Technology Choices

Amphion has three parts. A native iOS app records and plays audio, shows the tab, and keeps the tab in sync with playback. Firebase, a set of Google-managed cloud services, handles sign-in, the database of saved transcriptions, temporary audio uploads, and

notifications. A Python processing service in Google Cloud runs the music analysis: it separates the guitar and bass parts from the mix, detects the notes in each part, gives each note a string and fret, and finds the beats and bars, chord names, key, tempo, and tuning offset. It then writes the results to the database and deletes the uploaded audio (RC-03).

The analysis uses pretrained open-source models rather than a model trained from scratch. Because audio is deleted from the cloud after processing, all analysis runs in the cloud and none runs on the phone.

Technology	Rationale	Usage	Residency
Swift + SwiftUI	Native iOS framework with first-class access to Apple's audio tools; the project is iOS-only.	All screens, navigation, tab view with playhead and auto-scroll, chord diagrams, library.	User's iPhone
AVFoundation	Apple's built-in audio framework; supports recording and pitch-preserving speed change.	Microphone recording, file import, playback, 50-100% slow-down, A-B looping, and the playback position that drives tab sync (US-04, US-12 to US-14, RC-18).	User's iPhone
Firebase Authentication	Managed sign-in with built-in Apple, Google, and email/password providers.	Accounts and identity for every request (RC-01).	Firebase / Google Cloud
Cloud Firestore	Managed database with per-user security rules and live updates to the app.	Stores song records, job status, tabs (note lists per instrument), beats, chord names, song details, and user edits (RC-20).	Firebase / Google Cloud

Technology	Rationale	Usage	Residency
Cloud Storage for Firebase	Managed file storage protected by the same sign-in.	Temporary holding area for uploaded audio until processing finishes, and permanent private storage for custom song artwork (US-18).	Firebase / Google Cloud
Cloud Run functions (Python)	Event-driven Python with no server to manage; can hold the audio and ML libraries.	Starts when audio is uploaded, runs the analysis pipeline, writes results, deletes the audio.	Google Cloud
Demucs (6-part model)	Open-source pretrained model that splits a mix into separate parts, including guitar and bass.	Separates the guitar and bass parts before note detection (DD-16).	Inside the processing service
Basic Pitch (Spotify)	Open-source pretrained note-detection model that works on any instrument and finds several notes at once, with start and end times.	Detects the notes in each separated part (US-07).	Inside the processing service
Tab-assignment module (project code)	No off-the-shelf library applies Amphion's beginner rules; a small scored search over string and fret options is simple to write and test.	Picks a string and fret for every detected note (US-08, DD-15).	Inside the processing service
madmom	Open-source music-analysis library with pretrained models for beats, downbeats, chords, and key.	Beat and bar grid, chord names, key detection.	Inside the processing service

Technology	Rationale	Usage	Residency
librosa	Widely used Python audio library.	Audio loading, tempo (BPM) estimate, tuning-offset estimate.	Inside the processing service
Firebase Cloud Messaging	Standard push-notification service for iOS through Apple Push Notification service.	"Your song is ready" notification (US-06).	Firebase / Google Cloud

5.2 Architecture Diagram & Explanation

The app never talks to the processing service directly. It uploads audio to Cloud Storage and creates a song record in Firestore with status Processing. The upload triggers the processing function, which downloads the audio, separates the guitar and bass parts, detects notes and assigns strings and frets, runs the beat, chord, key, tempo, and tuning analysis, writes the tab and song details to Firestore, sets the status to Ready (or Failed), deletes the audio from storage, and sends a push notification. Because the app listens to its Firestore records, the library updates as soon as the status changes. During playback the app needs nothing from the cloud: it plays its local audio file and draws the tab from the saved results (RC-18, RC-19).

This design keeps all credentials and the ML code in the cloud (RC-06), keeps only results in the cloud (RC-03), and lets the analysis pipeline change without an app update.

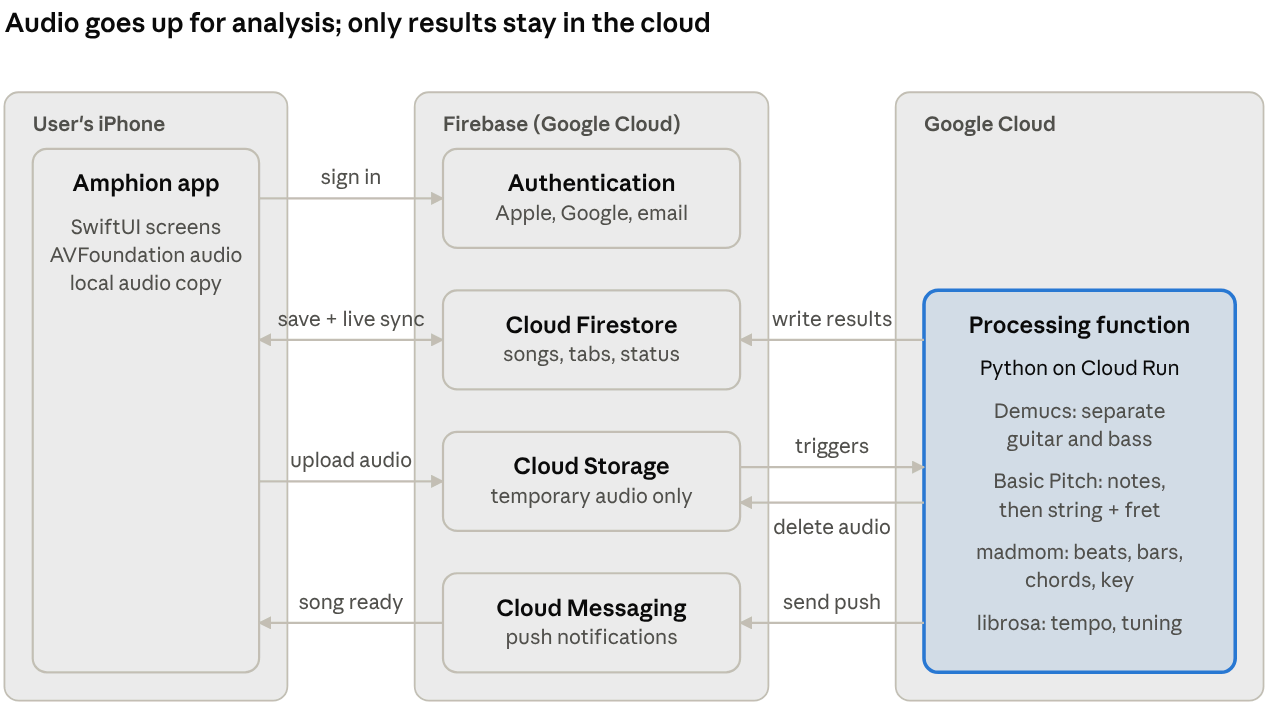


Figure 1: Architecture of Amphion

Figure 1: Architecture of Amphion. The highlighted processing function is the only component that touches audio content, and it deletes that audio when it finishes.

6 UI/UX Design

6.1 Application / Experience Flow

Amphion has one user role, so the flow is a single path. After signing in (US-01), the user lands on a bottom tab bar with three tabs: Library, Add Song, and Settings. Add Song leads to recording or uploading audio (US-04, US-05), then a review step where the user replays the audio, names the song, and can add an artist and choose a folder before submitting it. Submitting returns the user to the Library, where the song shows as Processing until it is Ready or Failed (US-06); a push notification announces when it is ready. The Library can be narrowed by folder, by search, and by a filter menu for date saved, song name, and artist (US-16, US-17).

Opening a Ready song shows the Song screen, which opens with a playlist-style header of the song's artwork, title, and artist (US-18). Below the header, the tab is the main view: guitar or bass tablature on the song's bars, with chord names in a small line above it (US-07, US-08), plus key, tempo, and tuning (US-09) and confidence markers (US-10). A

practice bar holds play/pause, speed, and loop controls; pressing play starts the song and the tab follows it with a moving playhead, highlighted notes, and automatic scrolling (US-12 to US-14). Tapping a note jumps playback there; pressing and holding a note opens a small editor to change, move, or delete it; tapping a chord name shows its finger diagram (US-08, US-11). Settings holds the guitar/bass instrument choice, appearance (System, Light, or Dark), and an Account screen for display name, email, password, sign-out, and account deletion (US-15, RC-15).

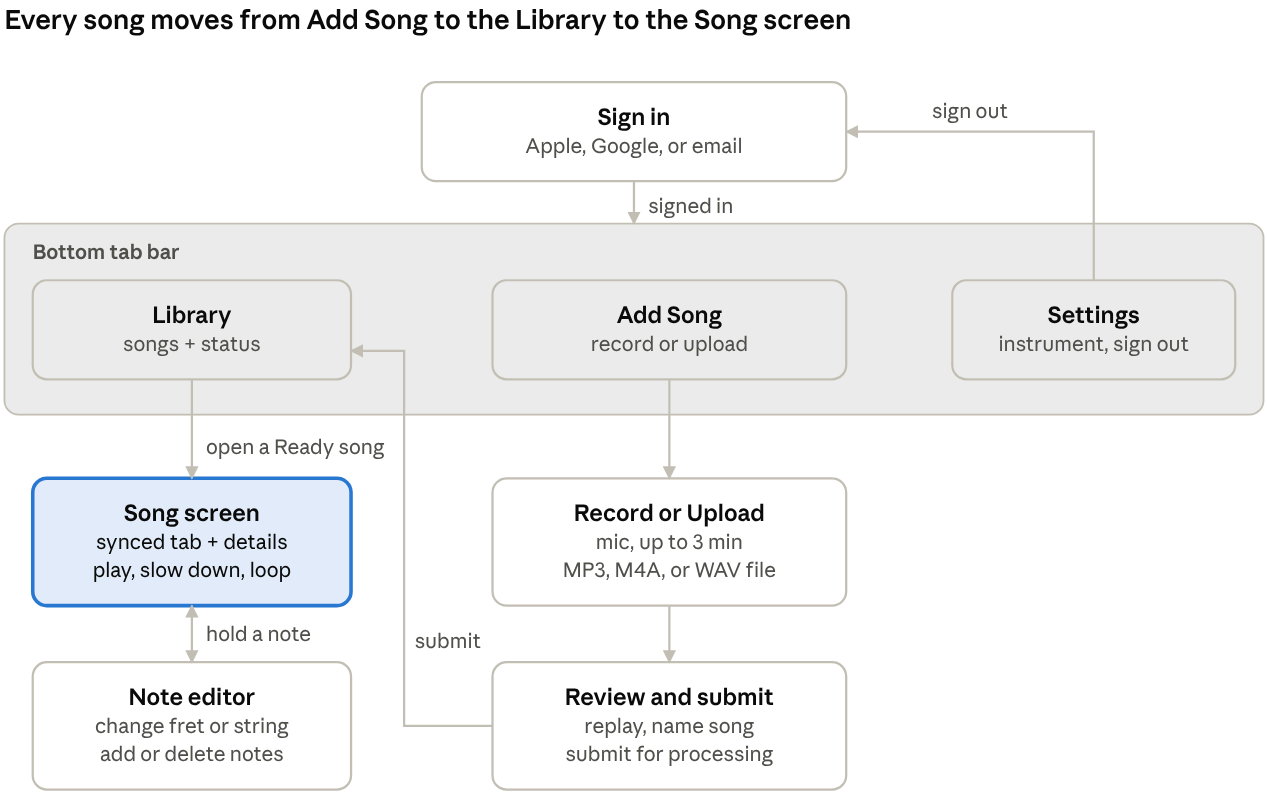


Figure 2: Major user/experience flow for Amphion

Figure 2: Major user/experience flow for Amphion. The highlighted Song screen is where users spend most of their time.

6.2 Interactive UI/UX Prototype

The interactive prototype will be built in Figma Make using this SDD and Figure 2 as its design context. The written SDD remains authoritative where the prototype and the requirements disagree.

Prototype / Figma Make Link: [Amphion interactive prototype (Figma Make)]

6.3 Design Alignment & Human Refinement

The prototype was reviewed against the user stories and constraints above. Each refinement below was made after reviewing the AI-generated screens and corrects a missing requirement or usability problem, not a cosmetic detail.

Design Refinement	Why It Was Changed	Related Story / Requirement
Added account details and a Delete account option to Settings, with a typed confirmation before deletion.	The AI design offered only Sign out. Apple requires in-app account deletion for apps that create accounts, and the SDD had missed it too, so US-15 and RC-15 were added.	US-15, RC-15, RC-01
Listed the supported file types and 3-minute limit on Add Song, and added a recording-tips card.	The first design gave no hint of what files are accepted, so users would learn only by having an upload rejected. The tips target lower phone-mic accuracy.	US-05, RC-07, RISK-03
Added bottom padding so the floating tab bar no longer covers the last Library card.	The tab bar hid part of the Failed song's Retry button, the one state where the user must act.	US-06, RC-09

As of v0.3, the prototype's Song screen still shows the earlier chord-chart design. It will be updated to the synced tab view (US-07, US-12), and the result recorded as a refinement above.

7 Key Design Decisions & Open Questions

7.1 Key Design Decisions

DD-01 - Tablature First, Chords as a Supporting Layer: The MVP's main output is note-level tablature: a string and fret for every note, on the song's beat grid. This reverses the v0.2 decision to ship chord charts first. Beginners want to play the actual part they hear, including riffs and bass lines, which a chord chart cannot show. Chord names stay as a secondary line above the tab, and the chord pipeline is kept because it is also the fallback

if tab accuracy misses its checkpoint (RISK-01). Note-level transcription is harder and less accurate than chord recognition, which is why parts are separated first (DD-16) and uncertainty is flagged (DD-06).

DD-02 - Pretrained Models Instead of Training a Custom Model: The pipeline uses existing open-source pretrained models (Demucs, Basic Pitch, madmom, librosa) rather than collecting data and training a new model. This keeps the ML risk inside a solo semester project and lets effort go into the user experience, the tab-assignment rules, and accuracy measurement (RC-10).

DD-03 - Cloud Processing with Temporary Audio: All analysis runs in the cloud, not on the phone. Audio is uploaded only for processing and deleted when the job ends (RC-03). This resolves the earlier open choice between on-device and cloud processing: cloud processing allows heavier models and pipeline updates without an app release, and deleting audio limits storage cost and copyright exposure.

DD-04 - Native SwiftUI Client: The app is built in SwiftUI rather than a cross-platform framework because the project is iOS-first and the core practice features (pitch-preserving slow-down, looping, recording) depend on Apple's AVFoundation audio framework. A future Android app would need a new client UI, but the backend and processing pipeline are platform-neutral and would be reused unchanged.

DD-05 - Upload-Triggered Pipeline: The app never calls the processing service directly. Uploading audio to Cloud Storage triggers processing, and results flow back through Firestore, which the app already listens to. This keeps credentials and ML code off the device (RC-06) and makes job status simple to show (US-06).

DD-06 - Show Uncertainty Instead of Hiding It: Low-confidence notes and chord names are flagged and editable (US-10, US-11, RC-11). Beginners cannot judge accuracy by ear, so the app must tell them where to double-check rather than present every result as correct.

DD-07 - Private-Only Library: Transcriptions are visible only to their owner and cannot be shared (RC-04). This keeps the MVP focused and avoids redistributing copyrighted material.

DD-08 - Capo Deferred, Tuning Reported as an Offset: Audio alone cannot show whether a capo was used, so capo suggestions are deferred (Section 2.3). Tuning is reported as how far the recording sits from standard pitch rather than as a named tuning.

DD-09 - Display Name Instead of Username: The account has an editable display name used only in the app's own greetings, and no username. Because the library is private-only (RC-04), no other user ever sees a name, so unique usernames would add uniqueness checks and taken-name handling with no user benefit.

DD-10 - Provider-Managed Credentials: Users who sign in with Apple or Google have no Amphion password; their email and password are managed by that provider. Only email/password accounts can change their password or email in the app, and those changes require re-entering the current password first.

DD-11 - One Folder per Song: Each song sits in at most one folder, like a file on a computer, rather than in several playlists at once. This keeps moving and deleting simple for beginners and keeps the data model to a single folder field per song. Multi-folder playlists could be added later without changing existing data.

DD-12 - Artist Entered by the User: Amphion does not identify songs or artists automatically. Song recognition would need a third-party audio-fingerprinting service with its own cost and licensing terms, and many target songs (original, local, or live recordings) would not be recognized anyway. Artist is an optional text field the user fills in on the review step or later, and it powers the artist search, filter, and sort (US-17).

DD-13 - Artwork Stored in the Cloud, Audio Is Not: Unlike song audio (RC-03), custom artwork is kept in Cloud Storage so it appears on every iPhone the user signs in on. Artwork is the user's own photo, small after compression, and carries none of the copyright concerns of song audio, so keeping it does not conflict with the audio-retention rule. Songs without custom artwork use generated abstract art, so the Library never shows blank cards.

DD-14 - The Audio Player Is the Only Clock: Synced playback reads the song position from the audio player on every screen refresh instead of starting a separate timer when play is pressed. A separate timer drifts, and it breaks as soon as the user pauses, scrubs, slows down, or loops. Because all tab data is stored in song time (RC-18), the same tab works unchanged at every speed and across every loop.

DD-15 - Beginner-Friendly String and Fret Choices: A detected pitch can usually be played in several places on the neck. The tab-assignment step scores each option, preferring open strings and lower frets, keeping notes that sound together within a reachable span, and avoiding big hand jumps between neighboring notes, then picks the best-scoring path through the whole song. These preferences are written as named, adjustable weights so they can be tuned against the benchmark (RC-10). This is not the deferred simplified-fingering feature: every detected note is still shown.

DD-16 - Separate Guitar and Bass Before Detecting Notes: Stem separation moves from stretch goal into the MVP pipeline. Note detection on a full mix picks up vocals, keys, and drums as false guitar notes, so the processing service first separates the guitar and bass parts and detects notes in each part on its own. The separated parts exist only inside the running job and are never stored (RC-03). The costs are extra processing time (RISK-02) and a single combined guitar tab when a song has more than one guitar part (RISK-06).

7.2 Open Questions, Assumptions & Risks

ID / Topic	Open Question, Assumption, or Risk
RISK-01 - Tab Accuracy (go/no-go checkpoint)	Note detection on separated parts may be much less accurate on full-mix recordings and phone-mic audio than on clean solo recordings. **Checkpoint:** 4 weeks after implementation starts, measure note accuracy on the 10-song benchmark (RC-10) as an F-measure, a single score that rewards finding the reference notes and penalizes extra wrong notes. The proposed pass bar is **60%** for guitar on uploaded files (OQ-07). **Fallback if missed:** make the beat-synced chord chart the main view again, keeping the same synced playback, playhead, and practice controls, and return note-level tabs to stretch-goal status. If chord accuracy (the percentage of beats where Amphion's chord matches the hand-checked chart) also misses its **70%** bar, narrow further to a single-chord identifier. A future agent must not silently lower either bar or switch to a fallback without this measurement.
RISK-02 - Processing Time and Cost	Stem separation and note detection add real work to the pipeline and may push a 3-minute song past the 2-minute target (RC-08), especially on CPU-only processing. Measure early; options include more memory per job, a GPU-backed service, or relaxing RC-08, with cost checked before switching.

ID / Topic	Open Question, Assumption, or Risk
RISK-03 - Recording Quality	Phone microphones and noisy rooms reduce accuracy more than clean uploaded files. Benchmark both input types separately, and show recording tips in the app if mic results lag.
RISK-04 - Library Compatibility	madmom's last packaged release is several years old and may not install cleanly on current Python versions, and Demucs and Basic Pitch each pin their own machine-learning library versions that may conflict with each other or with madmom. Verify early that all three install together in one container (installing from source or pinning versions if needed), and identify an alternative for any that cannot be used.
RISK-05 - User-Provided Audio	Users may process copyrighted recordings. Temporary processing, private-only results, and no redistribution (RC-03, RC-04) reduce but do not remove this risk; the Terms of Use linked from Settings state that users must have the right to use their audio (RC-16).
OQ-01 - Confidence Threshold	What confidence score counts as "low" for notes, and for chord names? Set each threshold by calibrating against the benchmark set so flagged items are actually the likely-wrong ones.
OQ-02 - Bass Tab (resolved)	Resolved in v0.3: the MVP shows a full bass tab made from the separated bass part (US-08), replacing the earlier plan of bass root-note positions.

ID / Topic	Open Question, Assumption, or Risk
OQ-03 - Second-Device Playback	Because audio is deleted from the cloud, a song opened on a different iPhone has its tab but no audio, so synced playback is not available there. Assumption for MVP: show the tab with a message that synced playback is available only on the original device (US-12, RC-19). Confirm with users.
OQ-04 - Full-Length Songs	Raising the 3-minute limit to full songs (about 6 minutes) is a goal by project end. Confirm processing time and cost at that length before changing RC-07.
OQ-05 - Tuning Display	A song in a lower key and a guitar tuned a half-step down can sound identical, so named tunings cannot be detected reliably. Decide how to word the tuning offset so beginners understand it (RC-13).
RISK-06 - Combined Guitar Parts	The separation model returns one guitar part, so songs with both rhythm and lead guitar, or acoustic and electric together, produce one combined guitar tab that can be crowded and hard to play. The MVP shows the combined tab; separate parts are a stretch goal (Section 2.3).
OQ-06 - Chord Display	How chord names and finger diagrams appear alongside the tab is being redesigned. Until then the SDD treats chords as a small line above the tab with a diagram on tap; a later version will update US-07, US-08, and 6.1.
OQ-07 - Tab Accuracy Bar	The 60% note-accuracy bar in RISK-01 is a proposal. Confirm it by running the pipeline on 2-3 benchmark songs early, since accuracy on clean solo recordings does not predict full-mix or phone-mic results.

8 References & Supporting Links

Platform and services

- SwiftUI documentation
- AVFoundation documentation and AVAudioUnitTimePitch (pitch-preserving speed change)
- Firebase Authentication, including Sign in with Apple on iOS
- Cloud Firestore
- Cloud Storage for Firebase
- Cloud Run functions
- Firebase Cloud Messaging

Music analysis

- madmom documentation and source repository (latest packaged release 0.16.1, November 2018; see RISK-04)
- librosa documentation
- Demucs source repository (music source separation; 6-part model includes guitar and bass)
- Basic Pitch source repository (note detection)
- mir_eval (standard note-accuracy scoring for the benchmark, RC-10)

Comparable products reviewed

- Klangio Guitar2Tabs
- Songsterr AI tabs from recordings (heise online)
- Moises on the App Store

Design method

- User Stories - Atlassian
- C4 Model

App Store policy

- Offering account deletion in your app - Apple Developer
- App Review Guidelines, 5.1.1(v) Account Sign-In - Apple Developer

8.1 Traceability Snapshot

ID	Design Intent	UI/UX Artifact	Architecture / Service	Later Planning Target
US-05	Upload an audio file	Add Song / Upload	Storage upload + upload-triggered processing function	Issue: validate and upload audio, create song record
US-07	Beat-synced tablature	Song / Tab view	Demucs + Basic Pitch + tab assignment; madmom beat grid; Firestore tab data	Issue: note and tab pipeline; tab view
US-10	Flag low-confidence notes	Song / Tab view markers	Confidence scores in Firestore tab data	Issue: confidence threshold calibration (OQ-01)
US-12	Tab plays in sync with the song	Song / Tab view playhead and auto-scroll	AVFoundation playback position drives the display (RC-18, DD-14)	Issue: sync engine, auto-follow, output-delay correction
US-13	Slow down without pitch change	Practice controls	AVFoundation time-pitch on device	Issue: practice playback engine
RC-03	Delete audio after processing	Not user-visible	Processing function deletes Storage object	Issue: audio cleanup on success and failure